

به نام خدا

گزارش کار آزمایش 6 – طراحی
انکوباتور – درس از طراحی سیستم
های دیجیتال

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)

1404/9/24

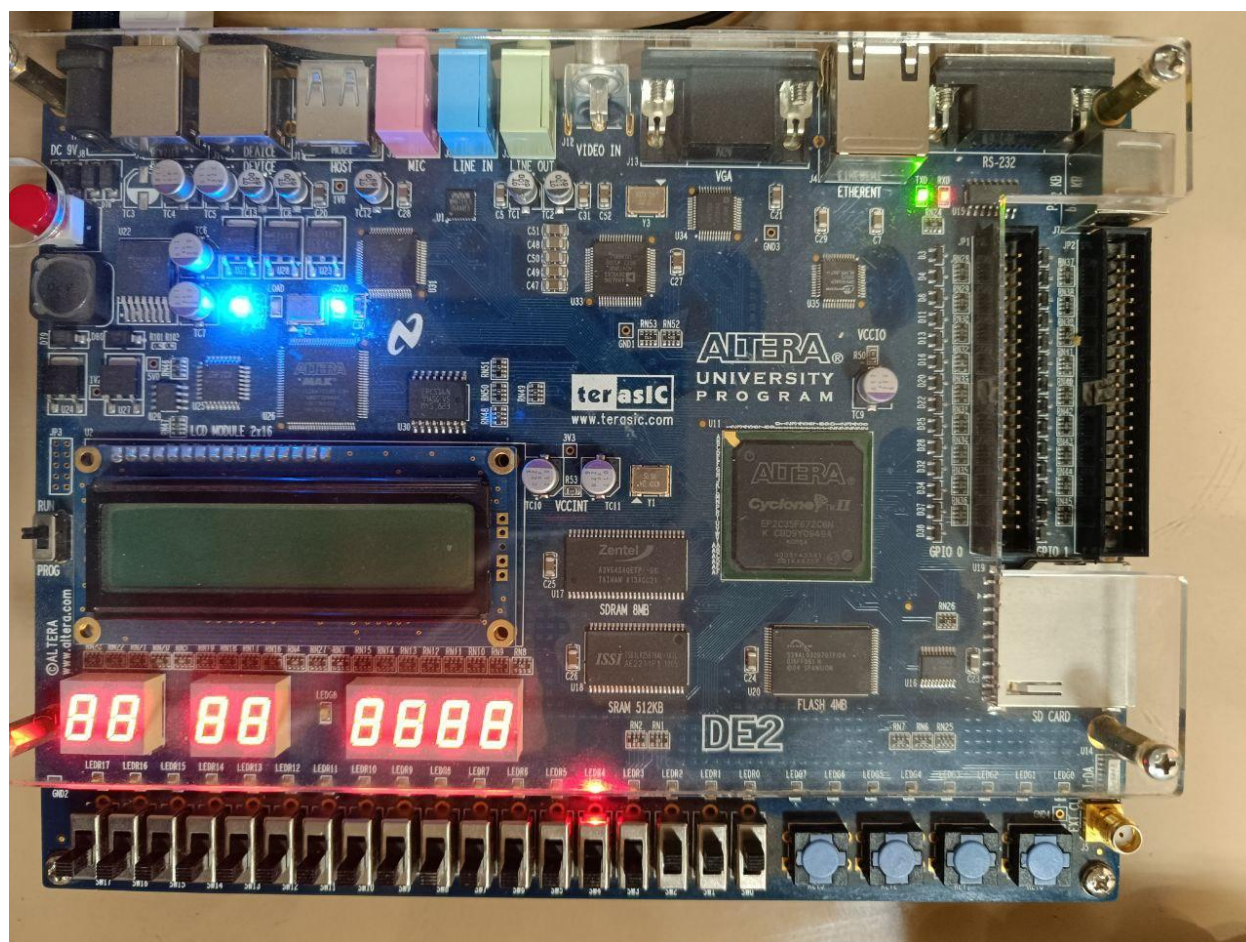
در ابتدا کدی که برای توصیف این آزمایش پیاده سازی کرده بودیم را تست کردیم.

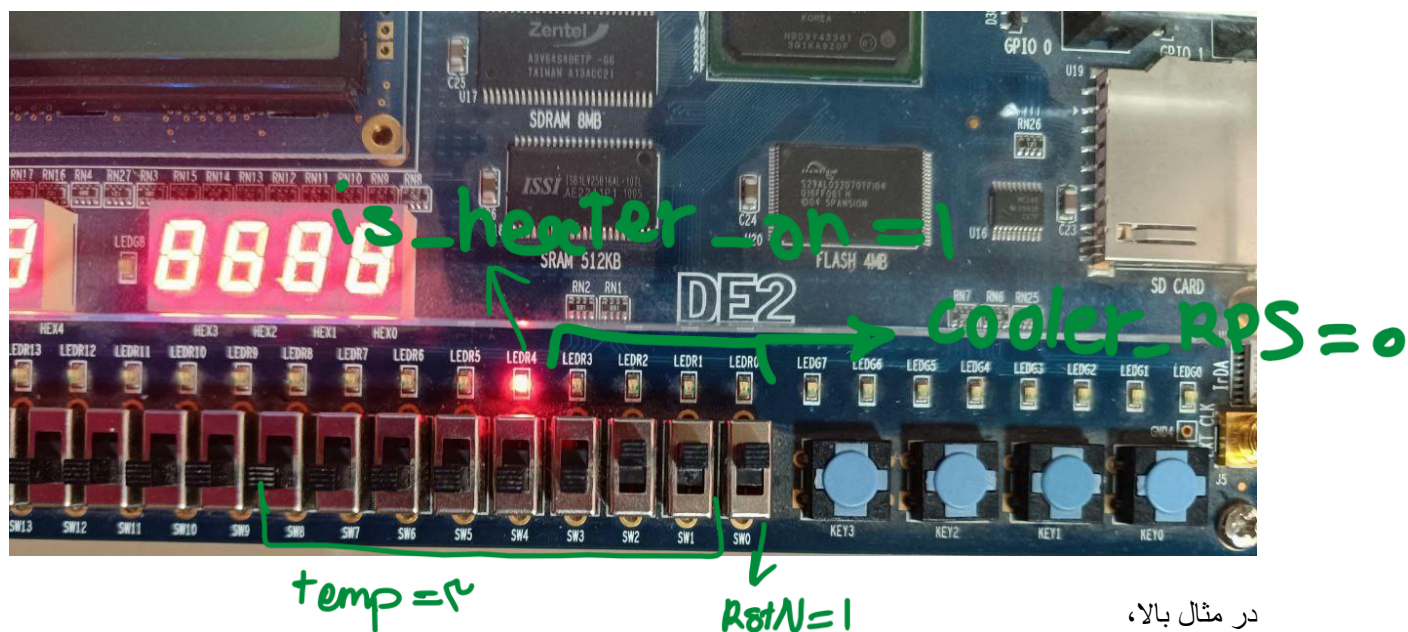
بعدش برای دیباگ و رفع اشکال کد، state را هم به عنوان خروجی قرار دادیم تا دقیقاً بفهمیم مشکل از کجاست.

بعد از آن متوجه شدیم مشکل از key1 است که کلاک را به آن وصل کرده بودیم (: در نهایت با وصل کردن کلاک به key2، کدی که برای این آزمایش پیاده سازی کرده بودیم، به درستی کار کرد.

پین‌ها: SW0 را به عنوان ورودی RstN و پین‌های SW1 تا SW8 را به عنوان بیت‌های ورودی دما قرار دادیم. همانطور که ذکر شد، KEY2 را هم به عنوان ورودی کلاک قرار دادیم. LEDR0 تا LEDR3 به عنوان cooler_RPS و LEDR4 را به عنوان is_heater_on قرار دادیم. برای دیباگ هم LEDR5 تا LEDR7 را به عنوان state قرار دادیم. (توجه کنید که همانطور که در آزمایشگاه هم عرض شد، این متفاوت با حالاتی است که در توصیف آزمایش ذکر شده و این به طور مفصل در پیش گزارش توضیح داده شده است.)

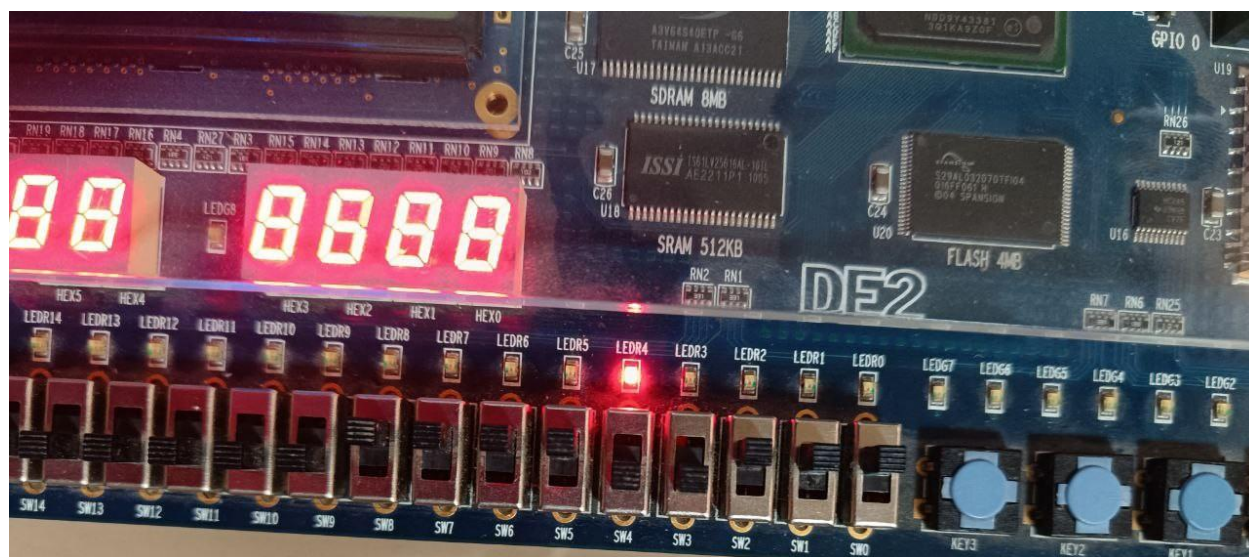
خب به نظر توضیح بیشتری باقی نمی‌ماند و کافی است برخی نتایجی که در آزمایشگاه عکس گرفتیم را در اینجا قرار دهیم:





در مثال بالا،

دما 3 است و مطابق انتظار $is_heater_on = 1$ است و طبیعتاً $cooler_RPS = 0$ است.



در مثال بالا دما را یک عدد منفی دادیم و مطابق انتظار نتیجه مشابه حالت قبل است.

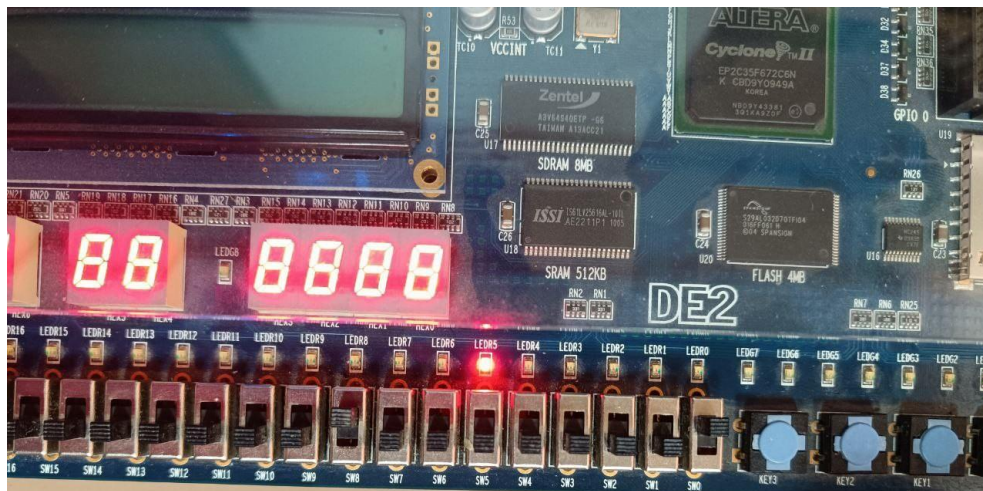
پس از اینکه برای دیباگ و پیدا کردن اشکال کد state را هم به عنوان خروجی نشان دادیم، نتایجی مشابه عکس زیر حاصل شد:

سپس $state = 0$ شده و این بلوک در نهایت اجرا می‌شود تا نتیجه ای که می‌بینیم حاصل شود:

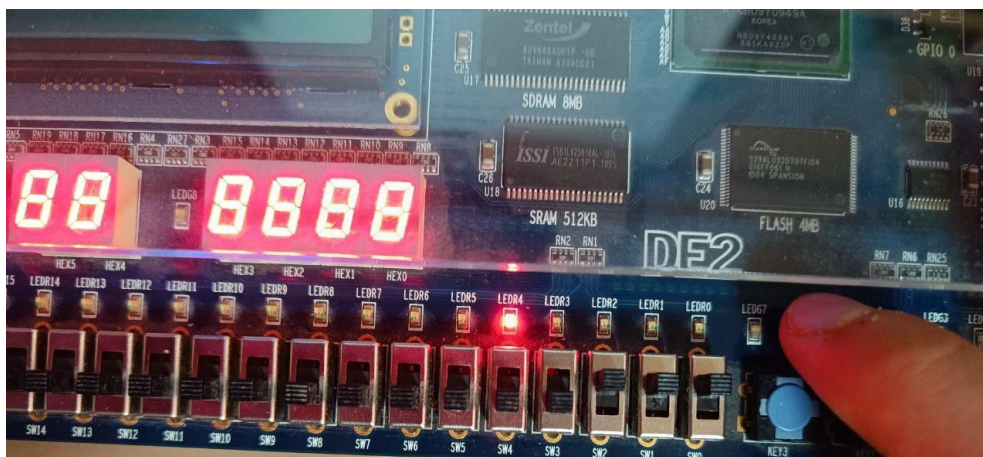
```
else if(state == 3'd0 && temp > 8'sd35) begin
    cooler_RPS <= 4'd4;
    state <= 3'd1;
end
```

پس از اینکه کد به درستی کار کرد، قرار شد که یک `error_flag` اضافه کنیم که اگر دما از محدوده مشخص شده خارج بود، این `flag` یک شود. نام این متغیر `invalid_temp` است.

مثلا در تصویر زیر، دما را یکی عدد منفی با مقدار بزرگ قرار دادیم (-128) و دیدیم که این `invalid_temp` به درستی یک شد. (SW5 تا SW7 مقدار `state` را نگه میداشت. این را پاک کردیم و SW5 را به عنوان `invalid_temp` قرار دادیم.)



یا مثلا در مثال زیر، دما به طور درست در محدوده تعیین شده است:

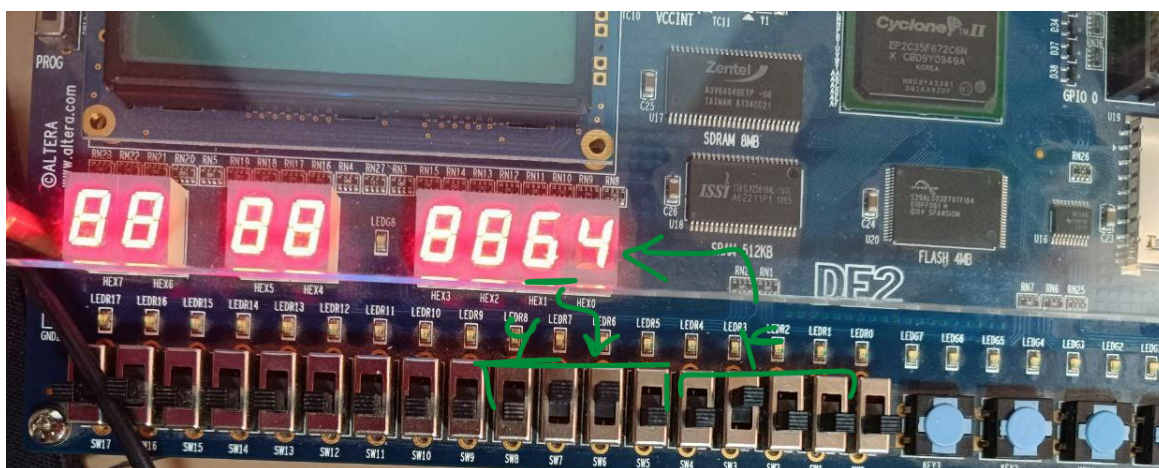
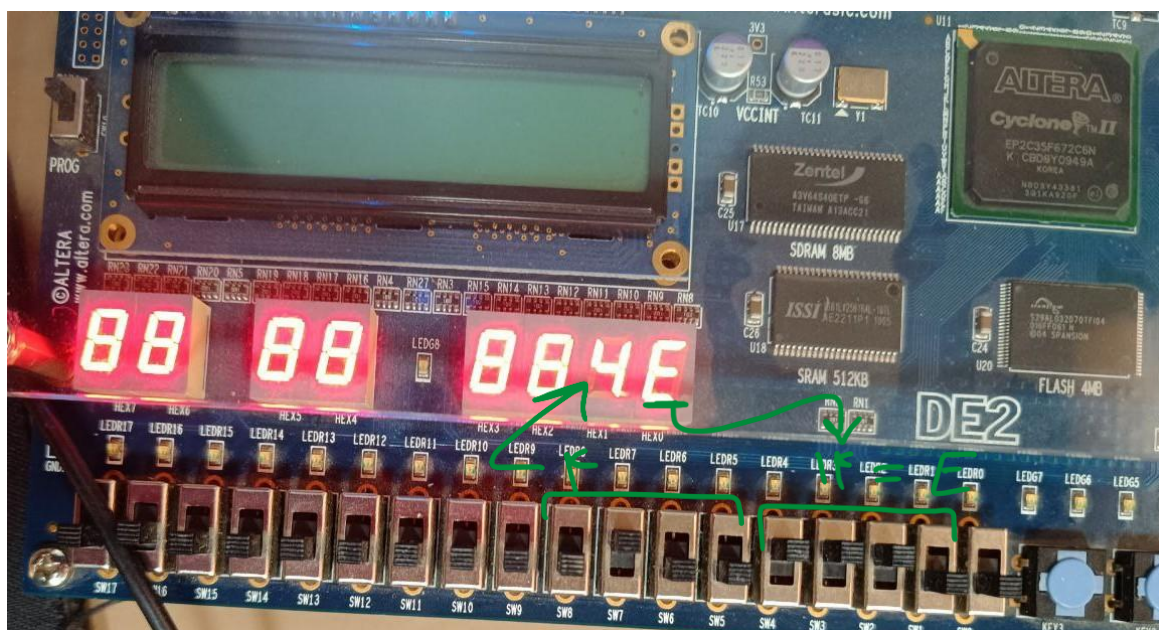


پس از این قرار شد یک task بسیار سخت را پیاده سازی کنیم:

اعداد رندوم تولید کنیم (!) سپس در یک stack بریزیم، سپس این اعداد را از stack بر میداریم و در یک seven segment نشان دهیم. این عددی که از stack بر میداریم دمایی ست که باید به عنوان ورودی به مدار بدهیم و مدار در هر بازه مشخص (مثلا یک دقیقه) state را تغییر دهد.

خب این یک کاری بود که باید انجام میدادیم اما بسیار پیچیده و زمان بر! به هر حال ما تلاش خود را کردیم... (:)

در ابتدا تلاش کردیم عدد دما را در seven segment نشان دهیم. صرفا قابلیت نشان دادن دما در seven segment. خدا رو شکر کد نشان دادن عدد در seven segment را قبلا پیاده سازی کرده بودیم. اما در اینجا نیاز بود که دو رقم را نشان دهیم و خب این کار را انجام دادیم:



این هم کدی ست که برای نشان دادن عدد در seven segment به کد اصلی اضافه کردیم (توجه شود که این کد را قبلا در آزمایش 4 استفاده کرده بودیم):

```
output wire [6:0] hex0,
output wire [6:0] hex1,
output reg invalid_temp
);

reg [7:0] stack_instance[3:0];
reg [2:0] pointer;
reg [2:0] state;
reg [7:0] temp;
wire [3:0] first_bits = temp[3:0];
wire [3:0] second_bits = temp[7:4];

assign hex0[0] = (first_bits == 4'd1 || first_bits == 4'd4 || first_bits == 4'd11 || first_bits == 4'd13);
assign hex0[1] = (first_bits == 4'd5 || first_bits == 4'd6 || first_bits == 4'd11 || first_bits == 4'd14 || first_bits == 4'd15);
assign hex0[2] = (first_bits == 4'd2 || first_bits == 4'd12 || first_bits == 4'd14 || first_bits == 4'd15);
assign hex0[3] = (first_bits == 4'd1 || first_bits == 4'd4 || first_bits == 4'd7 || first_bits == 4'd9 || first_bits == 4'd10 || first_bits == 4'd15);
assign hex0[4] = (first_bits == 4'd1 || first_bits == 4'd3 || first_bits == 4'd4 || first_bits == 4'd5 || first_bits == 4'd7 || first_bits == 4'd9);
assign hex0[5] = (first_bits == 4'd1 || first_bits == 4'd2 || first_bits == 4'd3 || first_bits == 4'd7 || first_bits == 4'd13);
assign hex0[6] = (first_bits == 4'd0 || first_bits == 4'd1 || first_bits == 4'd7 || first_bits == 4'd12);

assign hex1[0] = (second_bits == 4'd1 || second_bits == 4'd4 || second_bits == 4'd11 || second_bits == 4'd13);
assign hex1[1] = (second_bits == 4'd5 || second_bits == 4'd6 || second_bits == 4'd11 || second_bits == 4'd14 || second_bits == 4'd15);
assign hex1[2] = (second_bits == 4'd2 || second_bits == 4'd12 || second_bits == 4'd14 || second_bits == 4'd15);
assign hex1[3] = (second_bits == 4'd1 || second_bits == 4'd4 || second_bits == 4'd7 || second_bits == 4'd9 || second_bits == 4'd10 || second_bits == 4'd15);
assign hex1[4] = (second_bits == 4'd1 || second_bits == 4'd3 || second_bits == 4'd4 || second_bits == 4'd5 || second_bits == 4'd7 || second_bits == 4'd9);
assign hex1[5] = (second_bits == 4'd1 || second_bits == 4'd2 || second_bits == 4'd3 || second_bits == 4'd7 || second_bits == 4'd13);
assign hex1[6] = (second_bits == 4'd0 || second_bits == 4'd1 || second_bits == 4'd7 || second_bits == 4'd12);
```

در واقع دو خروجی 7 بیتی تعریف کردیم که متناظر با ارقام seven segment هستند. (hex0 , hex1) سپس دو متغیر هم تعریف کردیم که مقادیر 4 بیت راست و 4 بیت چپ دما را دارند (first_bits, second_bits). سپس بر اساس این دو متغیر منطق seven segment را پیاده سازی کردیم.

در مرحله بعد تلاش کردیم تا قابلیت استک را پیاده سازی کنیم. برای این کار لازم بود یک سیگنال controller اضافه کنیم. پیاده سازی ما چنین بود که اگر $control = 0$ باشد (و البته اگر استک پر نباشد)، عددی که در ورودی می‌دهیم را به درون stack پوش کن. اگر $control = 1$ باشد (و اگر stack خالی نباشد، یعنی $Empty = 1$ باشد) مقدار pop شده از استک را در متغیر دما (temp) بریز و در نهایت اگر $control = 2$ باشد، دمایی که از داخل استک pop شده است را به مدار اصلی (مدار اولیه incubator که حالات مختلفش پیاده سازی شده) بده.

متأسفانه در ابتدا تلاش می‌کردیم با control تک بیتی این کار را انجام دهیم، اما به مشکل می‌خوردیم و بعد متوجه شدیم که با control دو بیتی می‌توان این قابلیت را پیاده سازی کرد. این کد در نهایت سنتز شد:

← تست مربوط به push

← تست مربوط به pop

← منطق incubator نه در حال حاضر

۲ = control است.

(که با مقدار pop شده از استک کار میکنند)

```
else if(control == 2'd0 && !Full) begin
    stack_instance[pointer] <= input_temp;
    pointer <= pointer + 1;
    Full <= (pointer == 3'd7);
    Empty <= 1'b1;
end else if (control == 2'd1 && Empty) begin
    pointer <= pointer - 1;
    temp <= stack_instance[pointer-1];
    Empty <= !(pointer == 3'd1);
    Full <= 1'b0;
end else if (control == 2'd2) begin

    if(temp > 8'sd60 || temp < -8'sd10) begin
        invalid_temp <= 1;
    end

    else if(state == 3'd0 && temp < 8'sd15) begin
        state <= 3'd4;
        is_heater_on <= 1;
        invalid_temp <= 0;
    end

    else if(state == 3'd0 && temp > 8'sd35) begin
```

در ادامه قرار بود که اعداد تصادفی تولید کنیم. باید این قابلیت را پیاده سازی می کردیم. به نظر قابلیت ست که پیاده سازی آن آسان نیست.

متأسفانه فرصت نشد این قابلیت را پیاده سازی کنیم. اما مواردی که در آزمایشگاه در مورد تولید اعداد تصادفی در وریلاگ و استفاده از آن در FPGA یاد گرفتیم (و البته بعد آزمایشگاه) را به طور خلاصه می نویسیم:

در عمل ما باید اعداد شبه تصادفی تولید کنیم. بهترین و متداول ترین روش، روش LFSR است به معنای Linear Feedback Shift Register.

روش چنین است:

```
assign feedback = random[7] ^ random[5] ^ random[4] ^ random [3];
```

سپس در داخل بلوک always چنین چیزی می نویسیم:

```
if (rst) begin
```

```
    random <= 8'hA5;
```

```
end else begin
```

```
random <= {random[6:0], feedback};
```

```
end
```

چیزی که به random می‌دهیم در حالت reset، نقش همان seed را دارد که نباید صفر باشد.

این خلاصه‌ی تولید اعداد شبه تصادفی در وریلاگ به روش LFSR است که قابل سنتز است.